

Comenius University in Bratislava  
Faculty of Mathematics, Physics and Informatics

# Machine Learning for Generation of Graph of Given Degree and Girth

Bachelor Thesis

2026

Vladimír Jančár

Comenius University in Bratislava  
Faculty of Mathematics, Physics and Informatics

# Machine Learning for Generation of Graph of Given Degree and Girth

Bachelor Thesis

Study Programme: Applied Computer Science  
Field of Study: Computer Science  
Department: Department of Applied Informatics  
Supervisor: Mgr. Ján Pastorek

Bratislava, 2026  
Vladimír Jančár



## THESIS ASSIGNMENT

**Name and Surname:** Vladimír Jančár  
**Study programme:** Applied Computer Science (Single degree study, bachelor I. deg., full time form)  
**Field of Study:** Computer Science  
**Type of Thesis:** Bachelor's thesis  
**Language of Thesis:** English  
**Secondary language:** Slovak

**Title:** Machine learning for generation of graph of given degree and girth.

**Annotation:** Generating graphs with prescribed structural properties is a key task in combinatorial optimization, network design, and bioinformatics. Traditional algorithmic approaches (e.g., configuration model) often fail to find graphs with high girth and specific degree because the space of possible solutions is vast and the constraints are nontrivial.

**Aim:** The goal is to investigate potential applications of machine learning to problems in algebraic graph theory. The student will have the opportunity to engage with the state-of-art research in machine learning and algebraic graph theory and contribute to the field by generating new record graphs and training neural networks to predict algebraic properties.

**Literature:** Morris, C., Ritzert, M., Fey, M., Hamilton, W. L., Lenssen, J. E., Rattan, G., & Grohe, M. (2019). Weisfeiler and Leman Go Neural: Higher-Order Graph Neural Networks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01), 4602–4609. [<https://doi.org/10/ggfn97>] (<https://doi.org/10/ggfn97>)  
Paolino, R., Maskey, S., Welke, P., & Kutyniok, G. (2024). *Weisfeiler and Leman Go Loopy: A New Hierarchy for Graph Representational Learning*.  
Wu, L., Cui, P., Pei, J., & Zhao, L. (Eds.). (2022). *Graph Neural Networks: Foundations, Frontiers, and Applications*. Springer Nature Singapore. [<https://doi.org/10.1007/978-981-16-6054-2>] (<https://doi.org/10.1007/978-981-16-6054-2>)

**Supervisor:** Mgr. Ján Pastorek  
**Department:** FMFI.KAI - Department of Applied Informatics  
**Head of department:** doc. RNDr. Tatiana Jajcayová, PhD.  
**Assigned:** 25.02.2025

**Approved:**

Guarantor of Study Programme

---

Student

---

Supervisor

I hereby declare that I have written this entire bachelor's thesis independently, with the assistance of my thesis advisor, RNDr. Jozef Šiška, PhD., using the literature listed in the attached bibliography and artificial intelligence tools. I declare that I have used artificial intelligence tools in accordance with applicable laws, academic rights and freedoms, and ethical and moral principles, while maintaining academic integrity.

Bratislava, 2026

.....  
Vladimír Jančár

**Acknowledgments:**

I would like to thank my supervisor, Mgr. Ján Pastorek, for our weekly meetings and for the discussions of my progress throughout the work on this thesis. I also thank Samuel Varchol, who works under the same supervision on a different task, for the ideas we shared.

This work was supported by the use of computational resources of the supercomputer PERUN, operated by the Supercomputing Centre at the Technical University of Košice (TUKÉ), Slovakia with the support of the European Union from the funds of the Recovery and Resilience Plan of the Slovak Republic within the framework of project No. 17I03-04-P03-00001, Development and design of a supercomputer for the National Supercomputing Center.

The source code for this thesis is available at [github.com/Vl4dk0/gnn](https://github.com/Vl4dk0/gnn), and an accompanying interactive website at [vladimirjancar.sk](https://vladimirjancar.sk).

## Abstract

A  $(k, g)$ -graph is a  $k$ -regular graph whose girth is at least  $g$ , and the smallest such graph is a cage. Determining cage orders is a classical extremal problem, and the best known orders are still being improved by a mix of hand constructions and large computer searches. This thesis investigated whether graph neural networks (GNNs) and reinforcement learning (RL) are useful for constructing  $(k, g)$ -graphs, and if so where and how. Neither applies directly: a message-passing GNN cannot detect girth in general, and RL has worked on extremal-graph problems mainly where the space of moves already carries structure. The guiding question was therefore whether learning can guide where a search looks while exact algorithms decide which graphs are valid.

The work first examined what GNNs can learn about the two properties that define a  $(k, g)$ -graph, predicting a vertex's degree, a local quantity, and the length of its shortest cycle, a global one. It then worked through construction methods of increasing structure: direct edge editing under a learned policy, algebraic voltage lifts in which regularity is built in and only girth is searched, edge-swap refinement and excision that repair and shrink a graph, and Forge, a pipeline composing a voltage producer with refinement and excision.

Degree was predicted exactly by a model with a few thousand parameters, while the minimum cycle resisted every architecture, so girth was handled through explicit search. Across 22 targets, the voltage methods reached the most targets but produced graphs about twice the Moore lower bound on order, the classical searches were size-optimal but narrow, direct RL was weakest, and Forge was the most balanced, returning graphs near 1.23 times that bound and meeting the cage exactly on the smallest targets. Learned components rarely beat their non-learned counterparts. The role of learning here was to guide and densify a search whose validity exact algorithms still decide, rather than to replace structured search.

**Keywords:** graph neural networks, reinforcement learning, cage problem,  $(k, g)$ -graphs, voltage graph lifts, girth, heuristic search

# Contents

|     |                                               |    |
|-----|-----------------------------------------------|----|
| 1   | Introduction                                  | 1  |
| 2   | Background and Definitions                    | 2  |
| 2.1 | Graphs                                        | 2  |
| 2.2 | $(k, g)$ -Graphs and Cages                    | 2  |
| 2.3 | Groups                                        | 3  |
| 2.4 | Tabu Search                                   | 3  |
| 2.5 | Graph Neural Networks                         | 3  |
| 2.6 | Reinforcement Learning                        | 5  |
| 3   | Related Work                                  | 6  |
| 3.1 | Classical and Computational Cage Construction | 6  |
| 3.2 | Graph Neural Networks and Their Limits        | 7  |
| 4   | Predicting Graph Properties with GNNs         | 9  |
| 4.1 | Data Generation                               | 9  |
| 4.2 | Implementation                                | 9  |
| 5   | Construction Methods for $(k, g)$ -Graphs     | 11 |
| 5.1 | From Guided Search to Reinforcement Learning  | 11 |
| 5.2 | Direct Reinforcement Learning                 | 12 |
| 5.3 | Curriculum and Reward Design                  | 12 |
| 5.4 | Limits of the Direct Formulation              | 13 |
| 5.5 | Implementation                                | 13 |
| 6   | Voltage Graph Lifts                           | 16 |
| 6.1 | Parameters of the Construction                | 16 |
| 6.2 | Building the Lift                             | 16 |
| 6.3 | Computing Girth on the Base Graph             | 17 |
| 6.4 | Search Methods over Voltage Assignments       | 20 |
| 6.5 | Implementation                                | 20 |
| 7   | Refinement and Excision                       | 23 |
| 7.1 | Edge-Swap Refinement                          | 23 |
| 7.2 | Excision                                      | 26 |
| 7.3 | Implementation                                | 28 |
| 8   | Forge                                         | 31 |
| 8.1 | When to Refine a Lift                         | 31 |
| 8.2 | Implementation                                | 32 |
| 9   | Results                                       | 34 |

|                                              |    |
|----------------------------------------------|----|
| 9.1 What the Architectures Can Predict ..... | 34 |
| 9.2 Constructing $(k,g)$ -Graphs .....       | 35 |
| 9.3 Forge .....                              | 38 |
| 10 Future Work .....                         | 39 |
| 11 Conclusion .....                          | 40 |
| Bibliography .....                           | 42 |



## List of Figures

|           |                                                                     |    |
|-----------|---------------------------------------------------------------------|----|
| Figure 1  | One message-passing layer. ....                                     | 4  |
| Figure 2  | Depth and the receptive field. ....                                 | 4  |
| Figure 3  | A voltage lift of $K_4$ over $Z_3$ . ....                           | 17 |
| Figure 4  | How base cycles determine the girth of the lift. ....               | 18 |
| Figure 5  | A 2-swap. ....                                                      | 23 |
| Figure 6  | A 2-swap removing a short cycle. ....                               | 24 |
| Figure 7  | A 3-swap. ....                                                      | 24 |
| Figure 8  | Removing a radius-2 tree from the dodecahedral graph. ....          | 27 |
| Figure 9  | Excision of the dodecahedral graph down to the Petersen graph. .... | 27 |
| Figure 10 | Per-architecture accuracy: degree vs. minimum cycle. ....           | 35 |

## List of Tables

|         |                                                                                              |    |
|---------|----------------------------------------------------------------------------------------------|----|
| Table 1 | Degree prediction: best model per architecture. ....                                         | 34 |
| Table 2 | Minimum-cycle prediction: best model per architecture. ....                                  | 35 |
| Table 3 | Mean time to solve (seconds); blank = unsolved within budget. ....                           | 36 |
| Table 4 | Mean size of produced graphs, $ V  /  E $ (blank = unsolved). Columns as in<br>Table 3. .... | 37 |
| Table 5 | Forge by voltage producer, full pipeline, over all targets and seeds. ....                   | 38 |
| Table 6 | Stage ablation on the voltage-rl producer. ....                                              | 38 |

# 1 Introduction

Two distinct machine-learning approaches have proven powerful on graph problems in recent years. Graph neural networks learn representations of graph structure and now underpin systems such as the protein-structure models behind AlphaFold. Reinforcement learning instead searches the space of constructions directly: Wagner used a neural-network policy this way to build small graphs that disprove long-standing conjectures in combinatorics [1]. This thesis asks whether either can help with a hard construction problem from algebraic graph theory, building record graphs of prescribed degree and girth. The objects of interest are  $(k, g)$ -graphs: graphs in which every vertex has degree  $k$  and every cycle has length at least  $g$ . The smallest such graph is a *cage*, and determining its order is a classical extremal problem [2]. The definition fits in one line, yet building small  $(k, g)$ -graphs is very hard, and the best known orders are still being improved by a mixture of hand constructions and large computer searches [3].

Neither approach transfers to the cage problem as a black box. A message-passing graph neural network learns from local neighborhoods, and there is a precise sense in which it cannot even detect girth: its expressive power is bounded by the one-dimensional Weisfeiler–Leman test [4], and girth is provably beyond what such a model can compute [5]. Reinforcement learning, in turn, has succeeded on extremal-graph problems mainly when the space of moves already carries structure, not on unrestricted search over large graphs. The realistic question is therefore narrower: not whether a network can produce a cage by itself, but whether learning can *guide* where the search looks while exact algorithms decide which graphs are valid.

The recurring lesson from successful cage constructions is to constrain the search space mathematically and then explore only the freedom that remains. This thesis follows the same principle. It first probes what graph neural networks can and cannot learn about the two constraints that define a  $(k, g)$ -graph, and then works through a sequence of construction formulations of increasing structure, from unrestricted edge editing to algebraic voltage lifts in which regularity is built in and only girth is left to control. At each step the same question is asked: whether the learned component actually improves the search over its non-learned counterpart. The aim is to locate where, if anywhere, learning adds value beside structured search rather than to claim that it replaces it.

## 2 Background and Definitions

### 2.1 Graphs

A *graph* is a pair  $G = (V, E)$ , where  $V$  is a finite set of vertices and  $E$  is a set of edges. In the main construction setting, graphs are simple and undirected.

The *degree* of a vertex  $v \in V$ , denoted  $\deg(v)$ , is the number of edges incident with  $v$ . The *open neighborhood* of  $v$ , denoted  $N(v)$ , is the set of vertices adjacent to  $v$ .

A graph is  $k$ -regular if every vertex has degree  $k$ .

A *walk* is a sequence of vertices in which consecutive vertices are adjacent.

A *path* is a walk with no repeated vertices.

A *cycle* is a closed path of length at least 3.

A *closed walk* is a walk that starts and ends on the same vertex.

A closed walk is *reduced* if it never traverses an edge immediately followed by the same edge in reverse.

The *girth* of a graph, denoted  $\text{girth}(G)$ , is the length of its shortest cycle. If a graph has no cycles, its girth is treated as infinite.

A vertex  $v$  is *active* if  $\deg(v) > 0$ . The *active subgraph* of  $G$  is the subgraph induced by the active vertices, i.e.  $G$  with all isolated vertices removed. This is convenient when a construction algorithm carries a large fixed vertex pool but uses only the vertices it has already drawn into edges.

### 2.2 $(k, g)$ -Graphs and Cages

A  $(k, g)$ -graph is a  $k$ -regular graph whose girth is at least  $g$ . A  $(k, g)$ -cage is the smallest  $k$ -regular graph of girth  $g$ . Its order is commonly denoted  $n(k, g)$ . In search algorithms it is convenient to test the condition  $\text{girth}(G) \geq g$ , because this accepts every valid candidate for a target lower bound on girth.

The Moore bound gives a general lower bound on the order of a  $(k, g)$ -graph. It is useful both as a theoretical benchmark and as a practical target size for construction algorithms.

For  $k \geq 2$  and  $g \geq 3$ , the Moore bound is defined as follows:

$$2 \mid g \rightarrow M(k, g) = 1 + k \sum_{i=0}^{\frac{g-3}{2}} (k-1)^i.$$

$$2 \nmid g \rightarrow M(k, g) = 2 \sum_{i=0}^{\frac{g-2}{2}} (k-1)^i.$$

A graph meeting this bound is called a Moore graph. Moore graphs are rare. For most parameter pairs, the practical problem is therefore to close the gap between the Moore lower bound and the best published upper bounds.

## 2.3 Groups

A *group*  $(\Gamma, \cdot)$  is a set  $\Gamma$  with a binary operation  $\cdot$  that combines any two elements  $a, b \in \Gamma$  into an element  $a \cdot b \in \Gamma$ , subject to three axioms: the operation is *associative*, meaning  $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ ; there is an *identity* element  $e \in \Gamma$  with  $e \cdot a = a \cdot e = a$  for every  $a$ ; and every  $a \in \Gamma$  has an *inverse*  $a^{-1} \in \Gamma$  with  $a \cdot a^{-1} = a^{-1} \cdot a = e$ .

A group is *abelian* if its operation commutes, i.e.  $a \cdot b = b \cdot a$  for all  $a, b \in \Gamma$ . Otherwise it is *non-abelian*.

The *order* of a group is the number of its elements, and a group with finitely many elements is *finite*. The *order* of an element  $a$  is the smallest positive integer  $m$  for which  $a^m = e$ , the  $m$ -fold product of  $a$  with itself. The constructions in this thesis use finite groups, most often the *cyclic group*  $Z_n$ : the integers  $\{0, 1, \dots, n-1\}$  under addition modulo  $n$ , with identity 0 and the inverse of  $i$  equal to  $n-i$  modulo  $n$ .

## 2.4 Tabu Search

Tabu search is a local-search metaheuristic for minimizing a cost over a space of candidate solutions. From the current solution it considers the neighbors reachable by a small move and steps to one of them, repeating until the cost can no longer be lowered. To avoid becoming trapped, it may step to a neighbor that is temporarily worse when no improving move is available.

A *tabu list* is the short-term memory that makes this possible. It records the most recently applied moves, or their reversals, and forbids them for a few steps. This prevents the search from immediately undoing its last move and oscillating between the same few solutions, and it pushes the search to explore away from a local minimum instead of returning to it.

## 2.5 Graph Neural Networks

Ordinary feed-forward neural networks expect fixed-size vectors. Graphs do not have a fixed number of vertices, and their vertices do not have a canonical ordering. Graph neural networks address this by applying the same local update rule at every vertex. This makes the model independent of the input graph size and equivariant to relabeling of vertices.

Message-passing graph neural networks process graph-structured data by iteratively updating vertex representations through layers [6]. At layer  $t$ , each vertex  $v$  has a hidden representation  $h_v^t$ . The layer aggregates messages from the neighborhood  $N(v)$  and then updates the representation:

$$m_v^t = \text{AGG}(\{h_u^{t-1} : u \in N(v)\})$$

$$h_v^t = \text{UPD}(h_v^{t-1}, m_v^t).$$

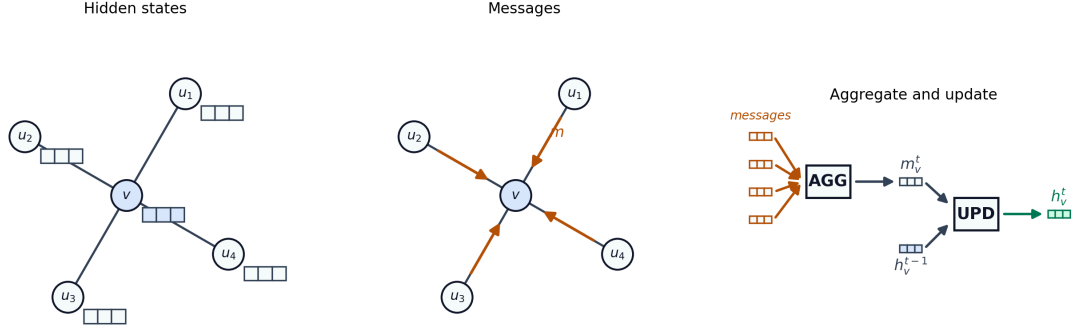


Figure 1: One message-passing layer.

Several parameters determine what such a model can express. The *number of layers* controls how far information can travel: after one layer, a vertex representation depends on its immediate neighbors, after two layers it can depend on vertices at distance two, and so on.

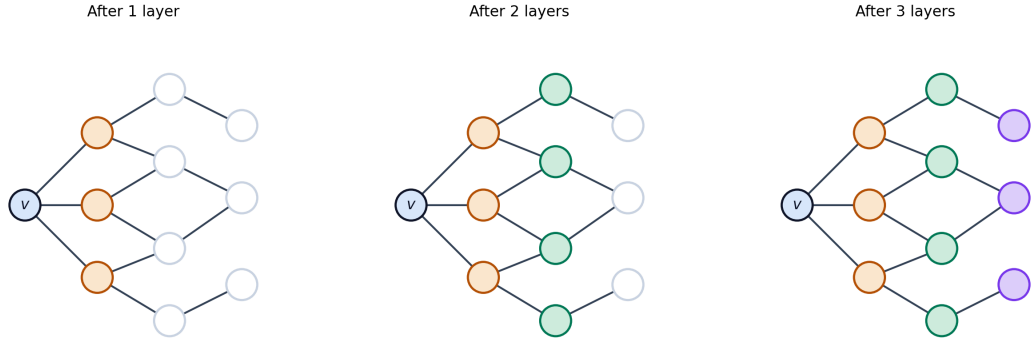


Figure 2: Depth and the receptive field.

The *hidden dimension* controls how much information can be stored in each vertex representation.

The *aggregation rule* determines which neighborhood statistics are easy to preserve, for example, normalized aggregation can make raw counting harder, while sum-like aggregation keeps count information more directly.

Edge features, positional encodings, and graph-level pooling become important when the task depends on edge labels or on the position of a vertex inside the whole graph. This is why local quantities such as degree are easier for standard GNNs than global quantities such as girth.

Grohe's survey on the logic of graph neural networks provides the theoretical framing for this limitation [4]. Standard message-passing GNNs are closely related to color refinement

and the one-dimensional Weisfeiler–Leman algorithm. This connection is important for the thesis because it explains why some graph properties are naturally accessible to ordinary GNNs while other global or symmetry-sensitive properties may require stronger architectures, additional features, or a different formulation of the search problem.

The experiments use several GNN architectures:

- *GCN*, a stable baseline based on degree-normalized neighbor aggregation [7].
- *GraphSAGE*, an inductive architecture based on learned aggregation [8].
- *GIN*, a sum-aggregation architecture with expressivity related to the Weisfeiler–Leman test [9].
- *GINE*, an edge-aware variant of GIN used when edge attributes i.e. weights of the edges, are part of the input [10].
- *GPS*, a graph transformer architecture combining local message passing with global attention [11]. GPS models often benefit from structural or positional encodings, such as random-walk-based encodings, because attention by itself does not identify a vertex’s structural role in the graph.
- *Loopy GNN*, a cycle-aware architecture [12]. Instead of aggregating only over a vertex’s direct neighbors, it aggregates over short paths that connect pairs of those neighbors without passing through the vertex itself, and each such path closes a cycle through the vertex, so the messages carry information about the local cycles a standard message-passing scheme cannot see.

## 2.6 Reinforcement Learning

*Reinforcement learning* is a way of learning to make decisions by interaction rather than from labeled examples. An *agent* observes the *state* of an *environment*, chooses an *action*, and receives a scalar *reward* together with the next state. The rule it follows, mapping each state to a choice of action, is its *policy*. One run from an initial state to a terminal state is an *episode*, and the agent’s goal is to maximize the *return*, the sum of rewards over an episode, optionally discounted by a factor  $\gamma \in (0, 1]$  that weighs nearer rewards more. Learning means adjusting the policy so that actions leading to higher return become more likely.

The policies in this thesis are trained with *Proximal Policy Optimization* (PPO) [13], a policy-gradient method that improves the policy from sampled episodes while a clipping term keeps each update close to the previous policy, so that training stays stable.

### 3 Related Work

The work builds on two lines of research: graph-theoretic construction of small regular graphs, and learned heuristics for difficult discrete search problems. The relevant background is not every known cage construction, but the recurring pattern behind successful methods: restrict the search space mathematically, then use computation to explore the remaining choices.

#### 3.1 Classical and Computational Cage Construction

The cage problem has been studied through algebraic constructions, finite geometries, Cayley graphs, voltage graph lifts, exhaustive generation, local search, and excision. The Dynamic Cage Survey provides the broad background and record-holder context [2].

The most relevant lesson from classical construction is that successful methods rarely search over arbitrary graphs. They usually impose structure first and search only inside the remaining degrees of freedom. Algebraic families illustrate this directly. Incidence graphs of finite projective planes and generalized quadrangles yield  $(q + 1)$ -regular graphs of girth 6 and 8 for prime powers  $q$ . Generalized hexagons give analogous constructions for girth 12. Cayley graphs on suitable groups produce highly symmetric candidates, and Ramanujan graphs [14] provide explicit algebraic families with optimal spectral properties and large girth.

These methods do not search adjacency at all. They encode the constraint into the algebraic object.

Voltage graph lifts continue the same pattern. A small base graph and a finite group define a covering graph whose regularity is inherited from the base. Exoo and Jajcay formalize the relation between voltages on closed walks in the base graph and the girth of the lift [15]. This reduces the search from  $O(n^2)$  adjacency choices on a lifted graph to a much smaller voltage assignment on the base.

A second family of methods relies on local search inside the space of  $k$ -regular graphs of a fixed order. Hill climbing and tabu search use edge swaps that preserve regularity and minimize a cycle-based cost function. Tabu lists prevent the search from immediately reversing recent moves. Excision methods take a known small cage, remove a carefully chosen subgraph (typically a tree or a small structured neighborhood), and reconnect the remaining deficient vertices to obtain a regular graph one to several vertices below the previous best [16]. Constraint programming and integer programming encodings have also been studied, but the size of the target graphs limits their direct applicability.



Recent computational work combines several of these ideas. Exoo et al. couple voltage-lift generation with advanced tabu search, hill climbing over base-graph and voltage choices, and tree excision, and report eleven new upper bounds on cage orders in a single pipeline [3]. The lesson for this thesis is methodological: improvements come from chaining structurally constrained search methods, not from any single algorithm.

These methods provide the natural non-learning baselines. A learned method is not very informative if it only beats unrestricted random edge editing. A stronger comparison asks whether learning improves a search process that already uses graph-theoretic structure, such as a voltage-lift search with exact short-cycle checks.

### 3.2 Graph Neural Networks and Their Limits

The machine-learning side is closest to work on graph neural networks and learned heuristics for combinatorial search, not only to graph generation. Work such as NeuroSAT showed that neural networks can learn useful signals on symbolic search instances even when they do not replace the exact solver [17]. The analogy here is that a learned component may guide which states or moves are explored first, while exact algorithms still verify the final graph.

Standard message-passing GNNs are well-suited to permutation-equivariant scoring of graph states [4]. Garg, Jegelka, and Jaakkola prove that local message-passing GNNs cannot compute several global graph invariants, including girth and diameter, and give the first data-dependent generalization bounds for such models [5]. This is precisely the regime of the cage problem: high girth, vertex-transitive or nearly so, and locally tree-like. On such graphs,  $L$ -layer message-passing GNNs collapse to nearly identical vertex embeddings, and standard architectures cannot resolve the global structure required to verify or score a candidate construction.

Two responses to this limit are relevant here. Subgraph-based architectures add explicit cycle or motif counts as structural features, which lifts expressivity above plain 1-WL message passing [18]. Cycle-aware hierarchies such as the Loopy framework [12] extend message passing with path neighborhoods that capture cycle structure. Both lines of work are used in this thesis as candidate architectures when the prediction target depends on cycle information.

Degree regularity, on the other hand, can be enforced by construction, and short-cycle violations can often be detected exactly. This motivates learned scoring and move ranking rather than replacing the mathematical verification step.

Reinforcement learning has also been applied directly to extremal graph problems. Wagner combined a cross-entropy method with a small neural policy to construct explicit

counterexamples to combinatorial conjectures from a single scalar objective [1]. The later RLGT framework formalizes the same setting with structural rewards [19].

Where reinforcement learning has succeeded on related problems, the action space already carried structure: Freire et al. apply it to the LDPC components of hypergraph product codes, where the search operates on a constrained construction rather than on arbitrary adjacency [20]. These results frame the construction experiments in this thesis. The unrestricted edge-editing formulation is the hardest case, and the algebraic formulations deliberately reintroduce structure.

## 4 Predicting Graph Properties with GNNs

Degree prediction and minimum-cycle prediction serve as controlled tests for the two structural constraints that define a  $(k, g)$ -graph. In both tasks the graph is given and the model assigns a label to each vertex.

Degree is a local property, a vertex degree can be recovered from the immediate neighborhood, so this task tests whether the model, data pipeline, and training loop can learn a simple structural quantity. If an architecture fails to predict degree reliably, it is not a good candidate for harder construction tasks without further tuning.

Minimum-cycle prediction is a harder structural task. For each vertex, the target is the length of the shortest cycle containing that vertex, with target 0 for vertices that do not lie on any cycle. Unlike degree, this cannot be determined from immediate neighbors alone. It requires information about paths that leave a vertex and later return to it. This makes the task closer to the girth constraint used in constructing  $(k, g)$ -graphs.

### 4.1 Data Generation

Both tasks use dynamically generated random graphs. Instead of training on a fixed set of graphs, new Erdos–Renyi graphs are sampled during training. This reduces the chance that a model simply memorizes a finite training set and gives a cheaper way to expose it to many small graph structures.

Labels are generated by exact graph algorithms. Degree labels are computed as  $\deg(v)$  for each vertex. Minimum-cycle labels are computed by searching for the smallest cycle containing each vertex. This keeps the supervision reliable even though the graphs themselves are random.

The input features are intentionally simple. The experiments use either a constant one-dimensional feature or a four-dimensional feature vector consisting of a normalized node index, two random real-valued coordinates, and one additional random scalar. These features should not be interpreted as meaningful graph attributes. They mainly give the neural network a vertex-level input tensor while forcing the model to learn from graph structure.

### 4.2 Implementation

Training samples a fresh random graph at every step and labels it exactly, rather than reusing a fixed dataset. Degree labels are immediate, but the minimum-cycle label is the interesting one.

For a vertex  $v$  we look, through each incident edge in turn, for the shortest way back to  $v$  that avoids that edge. Removing the edge to a neighbor  $u$  and finding a shortest path from  $u$

back to  $v$  gives a cycle through  $v$  one longer than that path; the shortest over all neighbors is the label, and a vertex on no cycle is labeled 0.

```
func min_cycle_label(G: Graph, v: Vertex):
    best: Integer = infinity;
    for each neighbour u of v in G:
        remove edge (v, u) from G;
        // the shortest detour back to v closes a cycle through this
edge
        d: Integer = length of the shortest path from u to v in G;
        if d is finite:
            best = min(best, d + 1);
        restore edge (v, u) to G;
        // a vertex on no cycle has no such detour
    if best is infinity:
        return 0;
    return best;
```

Algorithm 1: Exact minimum-cycle label for a single vertex.

## 5 Construction Methods for $(k, g)$ -Graphs

The target problem is to construct small  $(k, g)$ -graphs. This is harder than predicting a graph invariant, because the algorithm must actively choose edges while keeping the partial object close to a valid regular graph. The search space grows quickly with the number of vertices, and most arbitrary edge choices lead to invalid or unpromising graphs.

The problem is therefore more naturally viewed as a search problem than as a single prediction problem. A construction algorithm repeatedly holds a partial object, chooses a modification, checks which constraints are still satisfiable, and continues until it either reaches a valid graph or enters an unpromising region of the search space. Machine learning can enter this process in several different ways: as a learned heuristic for deterministic search, as a policy trained from complete construction attempts, or as a scoring function inside a constrained algebraic search.

### 5.1 From Guided Search to Reinforcement Learning

A natural first idea is to use deterministic search and let a GNN guide it. For example, one can build a graph edge by edge and use a learned scoring function to prioritize partial graphs that appear more likely to extend to a valid  $(k, g)$ -graph.

The difficulty is not the search loop itself. The difficulty is that ordinary supervised training does not apply: there is no straightforward way to label partial states well enough to train a model to predict whether to expand them. Positive examples can be obtained by taking subgraphs of known valid graphs. Useful negative examples are harder: a partial graph may look bad but still be extendable through further edits. Deciding whether a partial graph can be completed into a valid  $(k, g)$ -graph leads back to the same kind of problem. A supervised heuristic can therefore end up training mostly on easy examples far from the decision boundary.

This explains why a purely supervised A-star-style approach was not pursued as the main construction method.

The attractive scenario would be a good heuristic that guides A-star-style search without a pre-built dataset. Reinforcement learning provides exactly this: instead of asking for a dataset that says whether each partial graph is extendable, the model learns from complete construction attempts. The environment supplies rewards based on the observed consequences of actions: whether an edge edit was valid, whether the graph moved closer to regularity, whether short cycles were avoided, and whether the final graph satisfies the target constraints.

In this sense, the reinforcement-learning approach is not separate from search. It is a learned version of heuristic search. A hand-designed search algorithm chooses moves

according to a manually specified rule. An RL policy attempts to learn such a rule from repeated interaction with the construction environment.

It is important to keep in mind what this formulation does and does not provide. Individual illegal moves can be ruled out, as the next section describes: an addition that exceeds degree  $k$  or closes a cycle shorter than  $g$  is simply never offered to the agent, so every intermediate graph stays legal. What no such rule provides is completion. Unlike the algebraic constructions of the next chapter, where regularity is automatic, nothing here guarantees that a sequence of legal edits ever reaches a graph that is  $k$ -regular on enough vertices. The agent must learn to assemble one edge by edge rather than get stuck in a legal state that cannot be completed.

The direct-RL formulation tests how far a learned search can go when local legality is enforced but global completion is left to the policy.

## 5.2 Direct Reinforcement Learning

In the direct reinforcement-learning formulation, construction is represented as a sequential decision problem and trained with Proximal Policy Optimization [13]. The state is a partially constructed graph. Each action selects an unordered pair of vertices. If the edge is absent, the action attempts to add it. If the edge is present, the action attempts to remove it.

This direct graph-editing formulation has an advantage: it does not require a precomputed dataset of labeled partial graphs. The model learns from interaction. However, it also makes the action space and the reward design essential, and these are addressed below.

Several invalid or unhelpful actions are removed from the action space before the policy sees them, so the agent cannot select them at all:

- An edge cannot be added if either endpoint already has degree  $k$ .
- An edge cannot be added if it would create a cycle shorter than  $g$ .
- An edge cannot be removed if doing so would disconnect the active subgraph.
- An edge cannot be removed if it would leave too few active vertices to reach the Moore-bound target.

## 5.3 Curriculum and Reward Design

The motivation for the curriculum is direct: at uniform difficulty an untrained agent almost never produces a valid graph, so the learning signal collapses and nothing is learned. Starting from parameter pairs with a small Moore bound raises the per-episode success rate enough that the agent has something to learn from, and the difficulty can then be increased once that signal is reliable.

The environment orders parameter pairs by their Moore bound and unlocks the next pair only after the agent solves at least half of its last eight episodes on the current pair. Training starts from  $(3, 5)$ .

Curriculum learning is what makes the direct RL formulation learn at all. Without it, the agent never produces a valid  $(3, 5)$ -graph. With it, the agent reaches and solves the first stages, the  $(3, 5)$ -graph and the  $(3, 6)$ -graph. Reaching even these stages also depended on the reward design described next.

The environment uses a shaped reward. Earlier reward designs concentrated almost all signal at the end of an episode, and successful episodes were too rare for that signal to guide learning reliably. A per-step progress term was added so that the agent receives a positive signal when an action moves the partial graph closer to a valid construction and a negative one when it moves away. A valid final graph receives a large terminal reward, but the per-step term carries most of the learning signal.

The progress term is implemented as a potential-based shaping signal [21]. Let  $s$  denote a state (a partial graph) and let  $\Phi(s)$  denote its *potential*: a scalar that grows as the active subgraph approaches a  $k$ -regular graph on the Moore-bound number of vertices, rewarding both each active vertex approaching degree  $k$  and the total edge count approaching the corresponding  $kM/2$  edges.

The per-step shaping reward is

$$\Phi(s') - \Phi(s),$$

where  $s'$  is the state reached after taking the action in state  $s$ . The value is positive when the partial graph moves closer to the target and negative when it moves away.

## 5.4 Limits of the Direct Formulation

The direct graph-editing formulation is useful as a baseline, but it has structural weaknesses. Regularity is not guaranteed. It must be learned or enforced through action pruning and rewards. The action space also grows quadratically with the number of available vertices: on a graph with  $n$  vertices, there are  $\binom{n}{2}$  possible unordered pairs, and each pair can become an add-or-remove decision depending on the graph state.

These limits motivate the voltage-lift formulation of the next chapter, which builds regularity into the representation rather than leaving it to the agent.

## 5.5 Implementation

Two parts of the direct formulation need a concrete procedure: which edits the policy may make, and how each step is rewarded. The policy samples only from a precomputed set of legal edits, so an illegal one can never be selected. A missing edge is legal to add unless an

endpoint already has degree  $k$  or its endpoints are nearer than  $g - 1$  apart, which would close a cycle shorter than  $g$ . A present edge is legal to remove unless that disconnects the active subgraph or drops it below the Moore-bound vertex count.

```

func legal_edits(G: Graph, k: Integer, g: Integer):
    legal: Set of Vertex Pairs = empty;
    for each unordered pair (u, v) of vertices:
        if G has edge (u, v):
            // a removal must keep the active subgraph connected and
            // keep at least Moore-bound-many active vertices
            if removing (u, v) keeps the active subgraph connected
                and leaves at least Moore-bound-many active vertices:
                legal <- (u, v);
        else:
            // an addition must not exceed degree k and must not
            // close a cycle shorter than g
            if degree(u) < k and degree(v) < k
                and distance from u to v in G is at least g - 1:
                legal <- (u, v);
    return legal;

```

Algorithm 2: Building the set of legal edits offered to the policy.

The reward for a step is the change in a potential  $\Phi$  across the edit,  $\Phi(s') - \Phi(s)$ , where  $\Phi$  rises as the active subgraph approaches a  $k$ -regular graph on the Moore-bound number of vertices. It combines two normalized scores: how far the active vertices have progressed toward degree  $k$ , and how close the edge count is to that of the finished graph. A completed valid graph adds a large terminal bonus.



```

// k is the target degree, M the Moore-bound vertex count, g the target
girth
func potential(s: State):
    G = graph of state s;
    active = vertices of G with degree > 0;
    progress = sum over v in active of min(degree(v), k);
    // degree progress toward k-regular, in [0, 1]
    reg = progress / (k * M);
    // edge count toward the finished graph, in [0, 1]
    dens = min(edges of G / (k * M / 2), 1);
    return weighted sum of reg and dens;

func reward(s: State, s': State):
    r = potential(s') - potential(s);
    // bonus only when the edit completes a valid graph
    if s' is a valid (k, g)-graph:
        r = r + large terminal bonus;
    return r;

```

Algorithm 3: The potential and the per-step shaped reward.

## 6 Voltage Graph Lifts

Voltage graph lifts are an alternative construction formulation in which  $k$ -regularity becomes structural: every lift of a  $k$ -regular base graph is itself  $k$ -regular. The control problem then reduces to choosing group labels on the edges of a small base graph instead of choosing every edge of the large lifted graph, and the learned components introduced later in this chapter operate inside this reduced space.

### 6.1 Parameters of the Construction

A voltage lift is determined by three choices, and only three. Everything else is forced once they are fixed.

The first choice is a finite group  $\Gamma$ . Its role is to set how many copies of the base graph the lift contains: the lift consists of  $|\Gamma|$  copies of  $B$ , so a larger group produces a larger graph.

The second choice is an orientation of each base edge. This is only a bookkeeping convention for the direction in which a voltage is read. Reversing an arc and replacing its voltage  $a$  by the inverse  $a^{-1}$  produces exactly the same lift.

The third choice is the voltage assignment  $\alpha$ , which labels each oriented edge with an element of  $\Gamma$ . This is the only choice that affects girth. It is precisely the voltage assignment that the search and learning procedures set out to optimize.

What is not free follows from these three. The number of vertices is  $|V(B)| \cdot |\Gamma|$ , the degree of every lift vertex equals the degree of the base vertex it copies, and the adjacencies of the lift are fixed by the rule in the next subsection.

### 6.2 Building the Lift

The base graph  $B$  is small and may contain loops and parallel edges. For undirected graphs each edge is represented by two opposite arcs. If one direction carries voltage  $a$ , the reverse carries  $a^{-1}$ .

The lift has vertex set  $V(B) \times \Gamma$ . Writing  $(v, h)$  for the copy of base vertex  $v$  on layer  $h \in \Gamma$ , an arc from  $u$  to  $v$  with voltage  $a$  contributes, for every group element  $h$ , the edge

$$(u, h) \text{ to } (v, ha).$$

This thesis uses right multiplication by the voltage element. For abelian groups the order of multiplication is invisible, but it matters for non-abelian groups.

As an example, take  $B = K_4$  and  $\Gamma = Z_3$ , so the lift has  $4 \cdot 3 = 12$  vertices. Figure 3 shows an assignment in which a spanning star carries voltage 0 and the remaining triangle carries voltage 1. An edge with voltage 1 from  $u$  to  $v$  produces  $(u, h) - (v, h + 1)$  for  $h \in \{0, 1, 2\}$ , so each base edge becomes three lift edges and the three layers are joined cyclically. The resulting graph is 3-regular and connected.

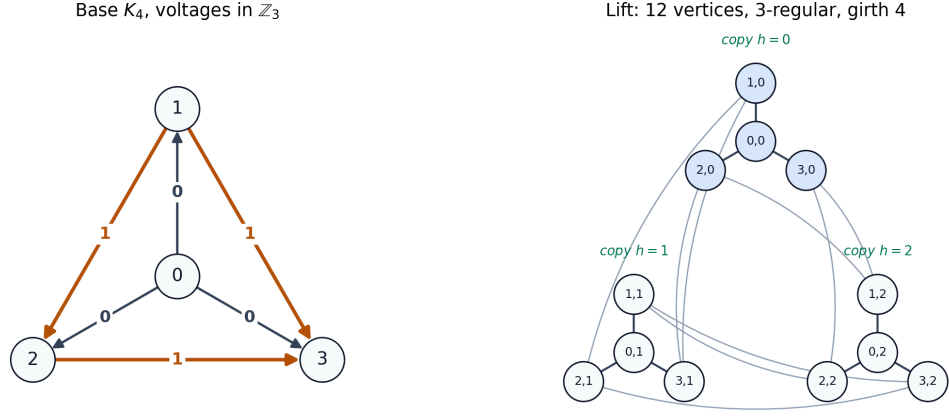


Figure 3: A voltage lift of  $K_4$  over  $\mathbb{Z}_3$ .

**Regularity is structural.** The reason the construction guarantees  $k$ -regularity is direct. Fix a base vertex  $u$  of degree  $k$  and a layer  $h$ . Each arc leaving  $u$  contributes exactly one lift edge at  $(u, h)$ : the arc to base neighbor  $v$  with voltage  $a$  produces the single edge  $(u, h) - (v, ha)$ . The  $k$  arcs at  $u$  therefore yield exactly  $k$  edges at  $(u, h)$ , regardless of which voltages were chosen, so every lift vertex has degree  $k$ .

This is the property that makes the formulation attractive for  $(k, g)$ -graphs. In direct edge-by-edge generation the agent must continually maintain the degree of every vertex. Here the degree constraint holds automatically, and only the girth constraint remains to be controlled through the voltage assignment.

### 6.3 Computing Girth on the Base Graph

The second advantage is that girth can be read off the base graph without building the lift. Consider a reduced closed walk  $W$  in the base graph. Multiplying the voltages along its arcs (using  $a^{-1}$  for an arc traversed backward) gives the *net voltage* of  $W$ .

Following the corresponding lift edges around  $W$  from layer  $h$  returns to the starting base vertex, but on layer  $hs$ , where  $s$  is the net voltage. Two cases arise:

- If  $s$  is the identity, the walk closes after one trip and lifts to a cycle of the same length as  $W$ .
- If  $s$  is not the identity, the walk must be repeated until the accumulated layer shift returns to the identity. The number of repetitions is the *order* of  $s$ , so  $W$  lifts to a cycle of length  $|W| \cdot \text{ord}(s)$ .

The girth of the lift is the length of the shortest cycle obtained this way:

$$\text{girth}(\text{lift}) = \min_W |W| \cdot \text{ord}(s),$$

minimized over reduced closed walks  $W$  of the base graph, where  $s$  is the net voltage of  $W$ , equivalently the shortest base walk whose net voltage is the identity.

Figure 4 illustrates both cases on an example. Every triangle of  $K_4$  has net voltage 1, which has order 3 in  $Z_3$ . A triangle therefore cannot close after one trip and instead wraps three times into a single 9-cycle that visits all three layers.

The 4-cycle 0–2–3–1, by contrast, has net voltage 0 and survives as an ordinary 4-cycle. It is the shortest surviving cycle, so the lift has girth 4, raising the girth of 3 in  $K_4$ . No voltage assignment on  $K_4$  over  $Z_3$  reaches girth 5. Higher girth requires a larger group or a different base graph, which is why the search later ranges over multiple base–group pairs.

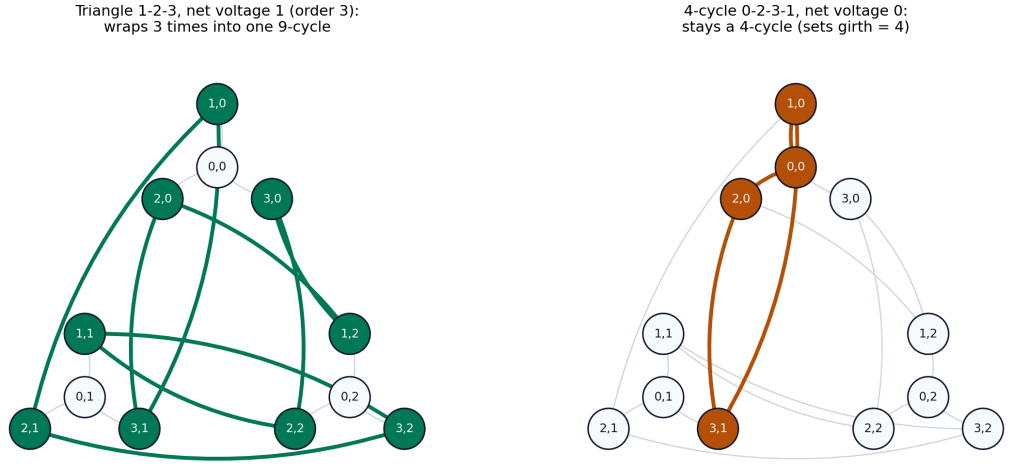


Figure 4: How base cycles determine the girth of the lift.

This analysis gives a cheap exact cost for search: enumerate the reduced closed walks shorter than the target girth  $g$  and count those with identity net voltage. The count is zero exactly when the lift has girth at least  $g$ , and it is computed on the small base graph rather than on the full lift. This exact cost is used by the search and [reinforcement-learning](#) procedures below, and it is the main reason the learned component can sit inside a constrained search instead of generating an entire graph from scratch.

**Reducing the search space.** The search becomes cheaper when fewer edges need a voltage: exhaustive or guided search over  $m$  undirected base edges scales with  $|\Gamma|^m$ , so any voltage that can be fixed in advance, without ruling out a single lift, shrinks the problem. Several base voltages can indeed be fixed for free, and this subsection explains which ones and why.

The reason such fixing is harmless is that the labels attached to the copies of a base vertex are arbitrary. At a vertex  $v$ , the  $|\Gamma|$  copies may be renamed among themselves. This is a pure relabeling of lift vertices, so the lifted graph is unchanged up to isomorphism.

Renaming the copies of  $v$  by a group element  $\tau(v)$  shifts the voltage of every arc touching  $v$ : it adds  $\tau$  at the arc's tail and subtracts it at the arc's head, so an arc from  $u$  to  $v$  acquires the new voltage

$$\alpha(u \rightarrow v) + \tau(u) - \tau(v).$$

Choosing one such element  $\tau(v)$  at every vertex, and letting it act on the voltages in this way, is called a *gauge transformation*.

A gauge transformation cannot change the girth, because it leaves the net voltage of every closed walk untouched. Adding up the corrections along a closed walk, each vertex it passes through is entered along one arc, contributing  $-\tau$ , and left along the next, contributing  $+\tau$ . The two cancel. Over the whole walk the corrections sum to  $\tau(\text{start}) - \tau(\text{end}) = 0$ . Since girth is determined entirely by the net voltages of closed walks, the lift is the same graph before and after.

This freedom can now be used to drive voltages to zero. Forcing a single arc to zero means choosing the potentials so that

$$\alpha(u \rightarrow v) + \tau(u) - \tau(v) = 0.$$

The limitation is that not every edge can be zeroed at once, and the reason is exactly the cancellation above: around any cycle the  $\tau$  corrections sum to zero, so the net voltage of a cycle is fixed, and a gauge transformation cannot touch it. If a cycle carries a nonzero net voltage, its edges therefore cannot all be made zero. Cycles are the obstruction, and only a set of edges that contains no cycle can be zeroed simultaneously.

This is what singles out a spanning tree. A spanning tree is the largest set of edges of a connected graph that contains no cycle, and it has  $|V(B)| - 1$  edges. Its voltages can always be cleared: root the tree, set  $\tau = 0$  at the root, and work outward.

Each tree edge joins an already-fixed vertex  $u$  to a new vertex  $v$ , and the choice  $\tau(v) = \alpha(u \rightarrow v) + \tau(u)$  makes that edge's voltage zero. Because the tree has no cycle, every vertex is reached exactly once, along a unique path from the root, so these choices never conflict. Adding any further edge would close a cycle, whose net voltage is beyond reach, so a spanning tree is as far as the zeroing can go.

What remains are the  $m - |V(B)| + 1$  non-tree edges, the only edges that carry genuine freedom. The effect is substantial. For the example,  $K_4$  has six edges and a spanning tree of

three, so three voltages are fixed to zero and only three remain to be chosen: the triangle on vertices 1, 2, 3 in the assignment above. The search assigns three voltages instead of six.

## 6.4 Search Methods over Voltage Assignments

Several search strategies share the voltage-lift formulation. They differ in how they choose voltage assignments, but they all keep exact verification at the end: a generated graph is accepted only after connectedness, regularity, and girth are checked.

For small groups and few base edges, exhaustive search enumerates every voltage assignment. This is only practical when  $|\Gamma|^m$  is small, where  $m$  is the number of undirected base edges. Random search samples assignments and verifies them. It is simple, but it is an important baseline because some small cases are easy enough that random search succeeds.

The tabu search follows recent computational work on small regular graphs [3]. It fixes tree-edge voltages to the identity, so only non-tree edges are searched. It then changes one voltage at a time and minimizes the number of short identity-voltage walks. A tabu list prevents the algorithm from immediately undoing recent moves.

The GNN-guided beam search assigns voltages edge by edge. A trained girth-predictor network scores partial assignments, and the search keeps only the highest-scoring candidates. *[Architecture and evaluation of this predictor to be detailed in Chapter 9.]* A meta-search procedure additionally enumerates choices of base graph and finite group. For cubic graphs, the candidates include the dumbbell base, a four-node cubic base, the prism base, and a Petersen-like base. Candidate groups include cyclic groups, dihedral groups, direct products, and semidirect products.

The reinforcement-learning environment treats one voltage assignment as one episode. The state is the base graph with partial voltage labels, the action is assigning a group element to an edge, and the episode length is the number of base edges. This is much shorter than the direct edge-editing environment. We have to use an architecture that works with edge features. That is why the policy model uses either a GINE-only encoder or a GPS encoder with GINE as the local message-passing component, followed by global pooling, an actor head over group elements, and a critic head for PPO.

## 6.5 Implementation

Because the search uses cyclic groups  $Z_n$ , a voltage is simply an integer in  $\{0, \dots, n-1\}$ , so a voltage assignment is a dictionary mapping each base edge to an integer, with the backward direction of an edge contributing the negated voltage modulo  $n$ . Two routines carry the chapter. The first reads the girth of a lift directly off the base graph: from every start vertex

it walks reduced closed walks, adding edge voltages modulo  $n$ , and the shortest walk that returns to its start with net voltage 0 is the shortest cycle of the lift.

```

func lift_girth(base: Graph, voltage: Dictionary of Edge to Integer,
               n: Integer, limit: Integer):
    best: Integer = infinity;
    for each start vertex of base:
        // each entry: current vertex, net voltage, walk length, last
        arc
        stack: Stack of (Vertex, Integer, Integer, Arc) = empty;
        stack <- (start, 0, 0, none);
        while stack is not empty:
            node, net, length, last = top of stack;
            remove top of stack;
            // a closed walk with net voltage 0 lifts to a cycle
            if node is start and net is 0 and length >= 3:
                best = min(best, length);
                continue;
            if length >= limit:
                continue;
            for each arc a leaving node:
                // stay reduced: never retrace the previous arc
                if a is the reverse of last:
                    continue;
                step: Integer = voltage[edge of a];
                // the backward direction carries the negated voltage
                if a runs against its edge:
                    step = (n - step) mod n;
                stack <- (head of a, (net + step) mod n, length + 1, a);
    return best;

```

Algorithm 4: Girth of the lift, computed by walk enumeration on the base graph.

The second routine is the tabu search that varies a voltage assignment toward the target girth. It fixes the spanning-tree edges to 0 and moves only the free, non-tree edges. Its cost reuses the same walk enumeration: the number of closed walks shorter than  $g$  with net voltage 0, which is exactly the number of short cycles that would appear in the lift. It changes one free voltage at a time to drive that count to 0, and a short tabu list keeps it from undoing a recent move.

```

// cost(voltage) = number of closed walks shorter than g with net
voltage 0,
func voltage_tabu(base: Graph, n: Integer, g: Integer, budget: Integer):
    free: List of Edges = the non-tree edges of base;
    // tree edges stay 0 (a fixed gauge); only free edges are searched
    voltage: Dictionary of Edge to Integer = 0 on every edge;
    for each edge e in free:
        voltage[e] = a random integer in 0 .. n - 1;

    tabu: Queue of (Edge, Integer) = empty;
    repeat up to budget:
        if cost(voltage) is 0:
            return voltage;

        // try every single-edge change and keep the one that helps most
        best_edge: Edge = none;
        best_value: Integer = 0;
        best_cost: Integer = infinity;
        for each free edge e:
            old: Integer = voltage[e];
            for each value c in 0 .. n - 1 with c != old:
                // a move is the pair (edge, new value); skip forbidden
                if (e, c) is in tabu:
                    continue;
                voltage[e] = c;
                trial: Integer = cost(voltage);
                if trial < best_cost:
                    best_cost = trial;
                    best_edge = e;
                    best_value = c;
                // undo the trial before testing the next value
                voltage[e] = old;

        // forbid undoing this move for a while, then commit it
        tabu <- (best_edge, voltage[best_edge]);
        voltage[best_edge] = best_value;
        drop the oldest tabu entry if the list is too long;
    return voltage;

```

Algorithm 5: Tabu search over free voltages, driving short cycles to zero.



## 7 Refinement and Excision

This chapter describes two methods that act on an already-constructed  $k$ -regular graph rather than building one from scratch. Edge-swap refinement edits a graph to remove short cycles while preserving its degree, and excision removes vertices from an existing graph to obtain a smaller one.

### 7.1 Edge-Swap Refinement

A voltage lift always returns a  $k$ -regular graph, but its girth may fall short of the target  $g$ , since a few short cycles can survive the construction. Such a candidate is close to a solution, and discarding it wastes the work already done. The refinement step instead edits a small number of edges to remove the surviving short cycles while keeping the graph  $k$ -regular.

The basic move is the 2-swap. Take two edges with no shared endpoint, say A–B and C–D on four distinct vertices, delete both, and reconnect the same four vertices the other way, for example as A–D and B–C (Figure 5). Each of the four vertices loses one edge and gains one, so every degree is preserved and a  $k$ -regular graph stays  $k$ -regular. Four vertices joined by two disjoint edges can be connected in three ways, so from the original pairing there are two alternative 2-swaps available.

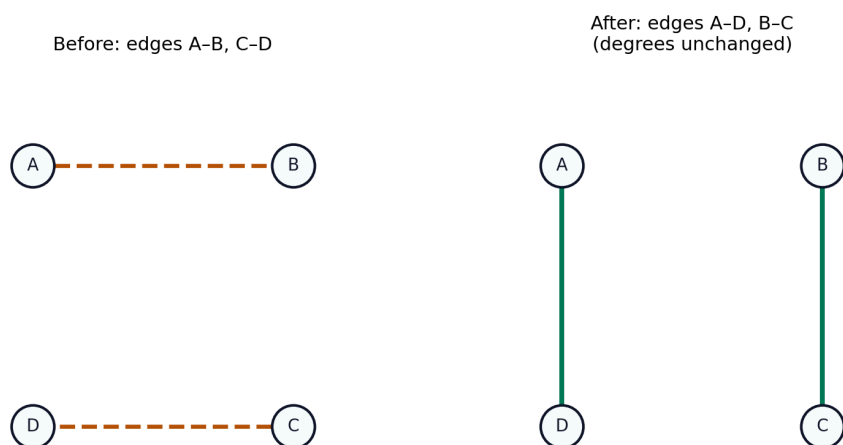


Figure 5: A 2-swap.

The two disjoint edges A–B and C–D (left, dashed) are replaced by A–D and B–C (right). Every vertex keeps its degree, so regularity is preserved.

A 2-swap can destroy a short cycle. To remove a triangle A–B–C, swapping its edge A–B against a disjoint edge D–E deletes A–B and breaks the triangle, again without changing any degree (Figure 6).

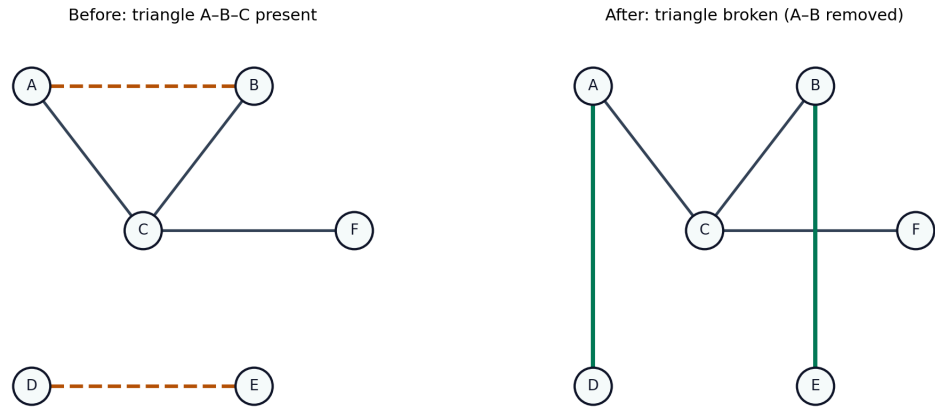


Figure 6: A 2-swap removing a short cycle.

The triangle A–B–C and a disjoint edge D–E (left) are swapped to A–D and B–E (right): the edge A–B is gone, so the triangle is broken, and all degrees are intact.

A larger move, the 3-swap, edits three edges at once. Take three pairwise-disjoint edges, on six distinct vertices, and rewired them into a different degree-preserving matching on the same six vertices (Figure 7). Like the 2-swap it leaves every degree unchanged, but it reaches configurations that no single 2-swap can produce.

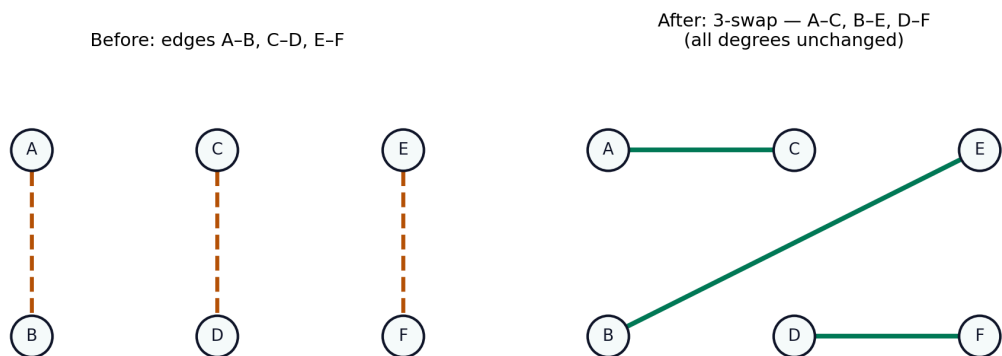


Figure 7: A 3-swap.

Three disjoint edges A–B, C–D, E–F (left) are rewired to A–C, B–E, D–F (right). All six vertices keep their degree.

A swap can remove one short cycle while creating another elsewhere, so these moves cannot be applied blindly. They have to be driven by a search that recognizes which swaps make progress.

**Search.** Progress means having fewer short cycles, so the search measures exactly that. Write  $N_c(G)$  for the number of cycles of length  $c$  in  $G$ . The simplest cost is the total number of cycles shorter than the target,

$$\sum_{c=3}^{g-1} N_c(G),$$

which drops by one whenever a short cycle is removed and reaches 0 exactly when no cycle shorter than  $g$  remains, the girth target. Because it counts every short cycle, the cost responds to each swap that removes one, giving the search feedback at every step.

Not all short cycles are equally harmful, however. A triangle must be removed before the girth can exceed 3, whereas a cycle just below the target is almost acceptable, so the search should remove the shortest cycles first. Weighting each length so that shorter cycles count for exponentially more makes this automatic:

$$f(G) = \sum_{c=3}^{g-1} w_c \cdot N_c(G), \quad w_c = 2^{g-c}.$$

A triangle then outweighs any single longer cycle, and the cost still reaches 0 exactly at the girth target.

The search minimizes  $f$  with [tabu search](#). At each step it applies the best available non-tabu swap: the one that lowers  $f$  most when some swap helps, and the least harmful move when none does. A [tabu list](#) forbids reversing the moves just made, which stops the search oscillating between two graphs, so taking a temporarily worse move to climb out of a local minimum does not pull it straight back. A tabu move is taken anyway when it beats the best graph seen so far.

The routine move is the 2-swap. When 2-swaps stop making progress (several iterations pass with no improvement while short cycles remain) the search escalates to a 3-swap, which can cross a barrier that traps the smaller move. Because there are  $O(|E|^3)$  candidate 3-swaps against  $O(|E|^2)$  2-swaps, the escalation is occasional rather than routine.

Scoring candidate swaps is the cost bottleneck. There are  $O(|E|^2)$  possible 2-swaps each iteration, and evaluating one exactly means recounting the short cycles it changes. A trained evaluator network replaces this with a cheaper estimate: a single pass of a [graph neural network](#) produces an embedding for every vertex, and the predicted change in cost for a swap is read from the embeddings of its four endpoints, together with a summary of the whole graph, through a small head.

The expensive graph reasoning is computed once per iteration and shared across all candidate swaps, which are scored together in one batched pass. The network uses cycle-count features and random-walk positional encodings, which break the symmetry that otherwise makes vertices indistinguishable on highly regular graphs.

Because the estimate is approximate, it is used only to rank candidates: the search verifies the most promising swaps exactly before committing to one. The approximation therefore affects only how quickly and how well the search proceeds, never the validity of the result, since every accepted graph is checked exactly against the girth target. This is the same division of labor as in the construction stage, where a learned component guides the search and exact verification decides.

## 7.2 Excision

A  $(k, g)$ -graph is often larger than it needs to be, and excision makes it smaller: from a valid  $(k, g)$ -graph it produces another one with strictly fewer vertices, pushing the order down toward the cage and the Moore bound. Balaban introduced the idea, removing a subtree from the  $(3, 12)$ -cage and suppressing the resulting degree-two vertices to obtain the  $(3, 11)$ -cage [22]. The step that makes excision hard is the repair that follows the removal, and our contribution is to drive that repair with a learned policy.

**Removing a tree.** Pick a root vertex and grow a breadth-first tree from it to depth  $d = \lfloor (g - 1)/2 \rfloor$ , then delete every vertex of that tree at once. This depth is the Moore radius: the deepest radius at which the breadth-first tree is forced to be acyclic, because two shortest paths from the root meeting within distance  $d$  would close a cycle of length at most  $2d \leq g - 1 < g$ , which the girth forbids. For odd  $g$  the surrounding ball can still carry a cycle of length exactly  $g$  (an edge between two leaves of the tree), but such an edge lies on no shortest path and is simply deleted with the tree, and removing vertices never lowers girth, so it does no harm. The vertices just outside the removed tree that lose a neighbour drop below degree  $k$ , and these are the deficient vertices. Figure 8 shows such a removal on the dodecahedral graph: a depth-2 tree of ten vertices is deleted, and the six boundary vertices it touched fall from degree 3 to degree 2.

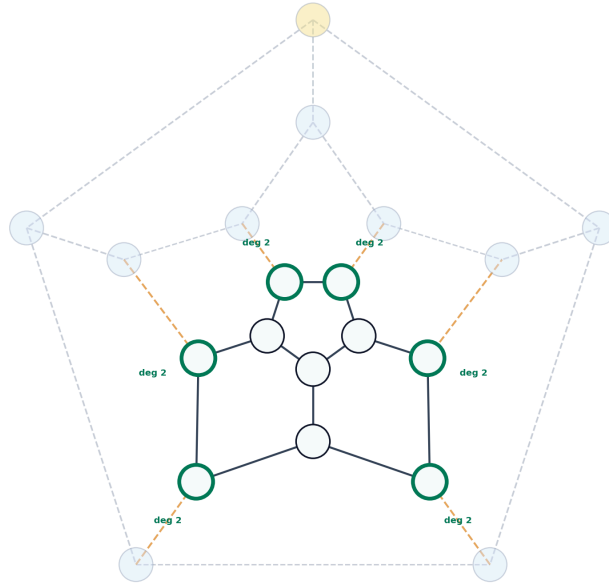


Figure 8: Removing a radius-2 tree from the dodecahedral graph.

**Stitching.** The deficient vertices must be sewn back up to degree  $k$ , and to preserve the girth an edge may join two of them only when they are at distance at least  $g - 1$ , so the cycle it closes is no shorter than  $g$ . Figure 9 carries this through: the six degree-2 boundary vertices of the dodecahedral graph are stitched with three such edges, restoring 3-regularity and producing the ten-vertex Petersen graph, the  $(3, 5)$ -cage, half the order we started from.

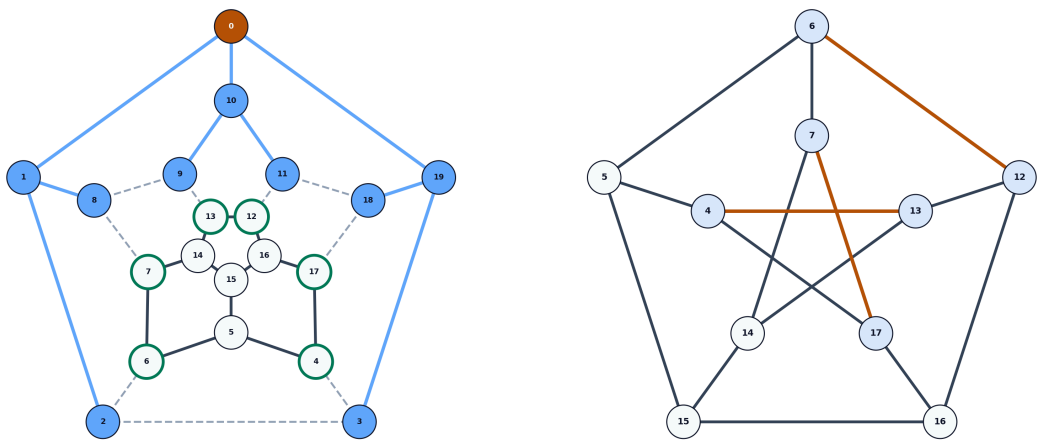


Figure 9: Excision of the dodecahedral graph down to the Petersen graph.

**The search.** A single removal-and-stitch may not succeed, since the boundary can be left with no legal way to reconnect, so excision is run as a search. Each vertex is tried as a root

in turn, and the first whose stitch yields a valid smaller graph (still  $k$ -regular and still of girth at least  $g$ ) is accepted. The search then restarts on the new, smaller graph and continues until no root admits a successful stitch. When the largest tree cannot be repaired a smaller depth is tried, and a root is skipped outright if removing its tree would drop the order below the Moore bound. The process therefore bottoms out at a cage, where no further excision is possible.

**A learned policy.** Choosing which deficient vertices to stitch, and in which order, is the hard combinatorial core, and it is where learning enters. We train a graph neural network actor–critic policy with reinforcement learning (PPO) that, at each step, reads the partially-repaired graph and picks the next girth-safe edge to add. Each vertex is described by its current degree and deficiency, the target  $k$  and  $g$ , counts of short cycles through it, and random-walk positional encodings, which break the symmetry that otherwise makes the vertices of a regular graph indistinguishable. The reward credits restoring a deficient vertex to full degree, with a bonus for completing a repair and a penalty for reaching a dead end. The policy is  $(k, g)$ -independent:  $k$  and  $g$  enter as features rather than as separate trained weights, so a single policy serves every target. When the policy reaches a dead end, an exact backtracking search takes over, so the policy guides the repair toward a solution while the exact search still finds one whenever its budget allows.

### 7.3 Implementation

The two edits in this chapter share one split of work: a learned or heuristic search proposes moves and exact checks decide. Refinement is a tabu search that, each iteration, embeds the current graph once and scores every candidate 2-swap from that shared embedding, applies the best non-tabu swap, and escalates to a 3-swap only when no 2-swap helps. Excision instead removes a tree and repairs the degrees it broke: the removed tree leaves deficient vertices, and  $k$ -regularity is restored by adding edges among those vertices alone, where an edge  $(u, v)$  is girth-safe only when  $u$  and  $v$  lie at distance at least  $g - 1$ , so the cycle it closes is no shorter than  $g$ . The repair is a backtracking search that always extends the most-constrained deficient vertex first.

```

func refine(G: Graph, g: Integer, budget: Integer):
    tabu: Queue of Swaps = empty;
    best: Graph = G;
    stale: Integer = 0;
    repeat up to budget:
        // f(G) is the weighted short-cycle cost, 0 means girth has
reached g
        if f(G) is 0:
            return G;
        embed G once with the evaluator network;
        candidates: List of Swaps = a sample of 2-swaps of G;
        score every candidate from its endpoint embeddings in one batch;
        rank candidates by predicted drop in f, largest first;
        // the learned score only ranks, the true f decides
        chosen: Swap = among the top-ranked few, the non-tabu swap with
the lowest true f (or a tabu one that beats f(best));
        if chosen exists:
            // taken even if it raises f, to escape a local minimum
            apply chosen to G;
            tabu <- chosen;
            drop the oldest tabu entry if the list is too long;
        if f(G) < f(best):
            best = G;
            stale = 0;
        else:
            stale = stale + 1;
        // when 2-swaps stop helping, cross the barrier with a larger
move
        if stale reached the escalation limit and f(G) > 0:
            big: Swap = best 3-swap that lowers f;
            if big exists: apply big to G;
            stale = 0;
    return best;

```

Algorithm 6: Tabu search for edge-swap refinement, with learned ranking of 2-swaps.

```

// deficit maps each deficient vertex to the number of edges it still
needs
func repair(H: Graph, deficit: Dictionary of Vertex to Integer, g:
Integer):
    if deficit is empty:
        return H;
    u: Vertex = deficient vertex with the fewest girth-safe partners;
    partners: List of Vertices =
        deficient vertices v != u with distance from u to v in H at
least g - 1;
    if partners is empty:
        return failure;
    for each partner v in partners, least-constrained first:
        add edge (u, v) to H;
        reduce deficit at u and at v;
        result = repair(H, deficit, g);
        if result is not failure:
            return result;
        // undo and try the next partner
        remove edge (u, v) from H;
        restore deficit at u and at v;
    return failure;

```

Algorithm 7: Girth-safe repair after excising a tree, by backtracking on the deficient vertices.



## 8 Forge

Each method built in the previous chapters reaches a different target and falls short on a different axis. The voltage lifts of Chapter 6 reach hard high-girth targets, but the graphs they return tend to be roughly twice the Moore bound, far from the cage. Classical search is size-optimal where it succeeds, yet its reach is narrow and it rarely closes the difficult targets at all. The refinement and excision of Chapter 7 do not construct anything on their own: refinement repairs a near-miss into a valid graph, and excision shrinks a valid graph toward the cage. Forge is the composition that lets each method do only what it does well. The producer supplies reach, refinement rescues lifts that are almost right, and excision recovers size-optimality.

Forge is a three-stage cooperative pipeline. The producer is a voltage-lift search that emits  $k$ -regular, connected lifts. Any of the voltage methods of Chapter 6 can drive it: the algebraic tabu search, its two GNN-guided variants that use a girth predictor or a tabu-cost predictor, or the reinforcement-learning voltage policy. Swapping the producer leaves the rest of the pipeline unchanged. The middle stage is the edge-swap refinement of Chapter 7, a tabu search over degree-preserving 2-swaps that escalate to a 3-swap when progress flattens, minimising a weighted count of cycles shorter than  $g$  in order to push girth up to the target. The final stage is excision, which takes a valid  $(k, g)$ -graph and shrinks its vertex count by removing girth-safe trees and repairing the boundary.

### 8.1 When to Refine a Lift

The idea that ties the stages together is to be generous about what the producer is allowed to pass on. A voltage search that insists on emitting only exact hits discards a great deal of nearly-finished work, because many lifts come out with girth just one or two below  $g$ . Forge instead admits these near-misses. A lift that already meets girth  $g$  is forwarded straight to excision. For the rest the question is whether refinement can realistically close the gap, and girth alone answers it poorly. Girth is the length of the shortest cycle, so a single surviving triangle drives it down to 3 even when the lift is otherwise almost right, and judging the lift by its girth gap would discard such a graph as hopeless when one edit would fix it. Forge instead measures how much of the lift is defective. An edge is defective when it lies on at least one cycle shorter than  $g$ , and the defective fraction is the number of such edges divided by the total, a number between 0 and 1. A lift is routed to refinement when that fraction is at most a small threshold  $\tau$ . A lift whose defective fraction exceeds  $\tau$  is dropped, since refinement is meant to repair a graph that is already close to correct rather than rebuild a badly defective one. Because the measure is a fraction of the lift’s own edges, it scales with graph size on its own, so the same  $\tau$  behaves consistently across small and large targets. The

cheap-to-produce lifts that the producer would otherwise throw away are thus rescued rather than wasted.

The measure works because it mirrors what refinement actually does. Refinement removes short cycles by rewiring edges, so the edges it must touch are exactly those lying on a cycle shorter than  $g$ , and the remaining edges supply the room to rewire them into. The defective fraction is therefore a direct estimate of how much repair a lift needs against how much intact structure is available to absorb it. A lift with only a few defective edges is a local repair that refinement usually completes, whereas a lift whose short cycles run through most of its edges would need a near-total rebuild that refinement cannot deliver.

The ordering of the three stages is forced by what each one needs from its input. Excision can shrink a valid graph but cannot close a girth gap, so it must come last, after the graph is already a valid  $(k, g)$ -graph. Closing the gap is exactly refinement's job, so refinement must sit between a producer that emits near-misses and the excision that consumes only valid graphs. The single ordering that respects these constraints is producer, then refinement, then excision, and the near-miss hand-off exists precisely to feed the slightly defective lifts into the one stage that can repair them. In the implementation the three stages run concurrently in a round-robin loop, and the first valid shrunk graph to come out of excision is the result.

Because the producer is interchangeable and both repair stages can be turned off independently, Forge is also a controlled way to ask which stage earns its keep. Disabling excision leaves a pipeline that only reaches valid graphs without minimising them, and disabling refinement leaves one that forwards only exact hits. Chapter 9 uses exactly these switches to isolate each stage's contribution and to compare the four producers on success rate and on graph order relative to the Moore bound.

## 8.2 Implementation

The three stages are chained into a single loop. Whenever the voltage search yields a  $k$ -regular lift, we branch on its girth. A lift already at girth  $g$  goes straight to excision. A lift below girth  $g$  is handed to edge-swap refinement only when at most a fraction  $\tau$  of its edges lie on a cycle shorter than  $g$ , and it continues to excision only if refinement reaches girth  $g$ . A lift whose defective fraction exceeds  $\tau$  is discarded, since refinement is not meant to rebuild a badly defective graph. Excision then shrinks a valid graph as far as it stays a valid  $(k, g)$ -graph. We keep the smallest valid graph found, and a surrounding harness repeats the loop in parallel to search for record graphs.

```

func forge(g: Integer, tau: Real):
    // smallest valid (k, g)-graph found so far
    best: Graph = none;
    repeat:
        candidate: Graph = next k-regular lift from the voltage search;
        gc: Integer = girth of candidate;
        if gc < g:
            // near-miss: refine only when few edges are defective
            if defect_fraction(candidate, g) > tau:
                continue;
            candidate = refine candidate toward girth g;
            // refinement may fail to reach girth g
            if girth of candidate < g:
                continue;
        // now a valid (k, g)-graph, shrink it
        candidate = excise candidate while it stays a valid (k, g)-
graph;
        if best is none or candidate has fewer vertices than best:
            best = candidate;
    return best;

```

Algorithm 8: The Forge pipeline: voltage search, near-miss refinement, then excision.

## 9 Results

All models were trained on the PERUN supercomputer at the Technical University of Košice. Evaluation ran every method under a 60-second search budget per attempt on a single PERUN node. The node has 256 CPU cores, and the benchmark ran 128 parallel workers. At 256 workers success collapsed: each worker leaves its numerical-library threads unpinned, so 256 processes oversubscribe the 256 cores and starve the scheduler, and because the budget is wall-clock, each attempt then completes fewer search iterations. The 128-worker setting leaves headroom. Throughout, higher accuracy and success are better, while lower MAE, time, and graph size are better.

### 9.1 What the Architectures Can Predict

The two properties that define a  $(k, g)$ -graph sit at opposite ends of locality: degree is a one-hop count, while the length of the shortest cycle through a vertex depends on the whole structure. Each architecture is evaluated on both tasks and scored by exact-rounded accuracy and mean absolute error.

| Architecture | Params  | Accuracy $\uparrow$ | MAE $\downarrow$  |
|--------------|---------|---------------------|-------------------|
| SAGE         | 4 900   | 1.000               | 0.00              |
| GPS–GIN      | 43 986  | 0.998               | 0.00              |
| GPS–SAGE     | 43 854  | 0.980               | 0.02              |
| GIN          | 151 948 | 0.887               | 0.34              |
| Loopy        | 620 977 | 0.512               | $5.3 \times 10^5$ |
| GPS–GCN      | 39 758  | 0.457               | 1.03              |
| GCN          | 2 692   | 0.382               | 1.17              |

Table 1: Degree prediction: best model per architecture.

Degree is solved. A SAGE model with 4900 parameters reaches perfect accuracy, and GPS–GIN matches it within rounding. The degree-normalised GCN variants are weakest, since dividing each message by neighbour count discards the very count being asked for. Loopy reaches moderate accuracy, but its regression head is numerically unstable and its MAE blows past  $10^5$  on the vertices it mispredicts.

| Architecture | Accuracy $\uparrow$ | MAE $\downarrow$ |
|--------------|---------------------|------------------|
| Loopy        | 0.510               | 1.45             |
| GPS-GCN      | 0.311               | 1.43             |
| GCN          | 0.227               | 1.80             |
| SAGE         | 0.119               | 1.97             |
| GPS-GIN      | 0.068               | 3.91             |
| GPS-SAGE     | 0.068               | 3.97             |
| GIN          | 0.055               | 16.18            |

Table 2: Minimum-cycle prediction: best model per architecture.

Minimum cycle is solved by nothing. The best model is the cycle-aware Loopy at 0.510 accuracy, and the ranking inverts: SAGE and the GIN-based models, strong on degree, are weakest here. A model can master the local count yet fail on the cyclic structure, so girth is not recoverable as a supervised per-node label and must be handled by explicit search in the construction methods.

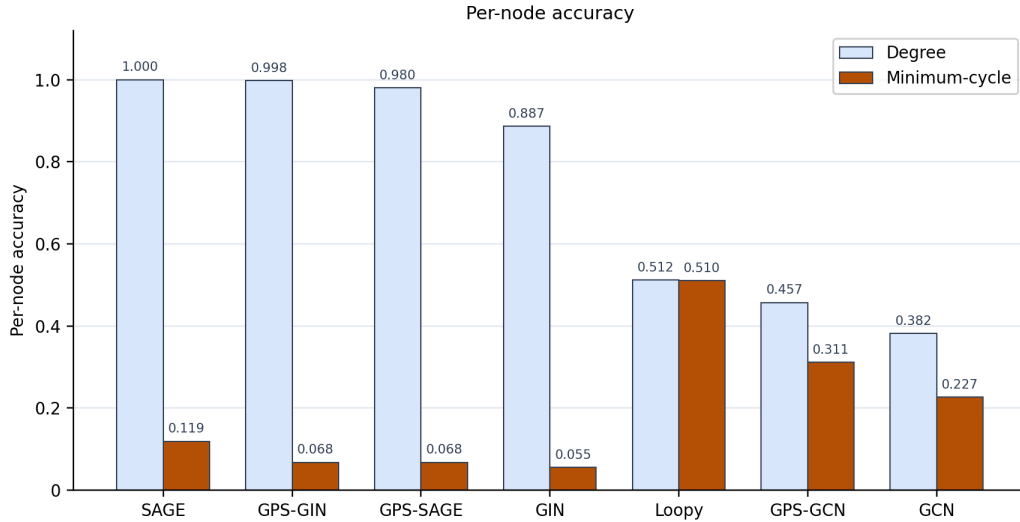


Figure 10: Per-architecture accuracy: degree vs. minimum cycle.

## 9.2 Constructing (k,g)-Graphs

The construction methods are compared on 22 targets, 100 seeds each, 60 seconds per attempt. Lower time and smaller graphs are better, and a blank cell means no seed solved that target within budget, so the filled cells double as the coverage map. In the columns, v-rl is the reinforcement-learning voltage search, v-girth, v-tabu, and v-alg are voltage searches guided by a girth predictor, a tabu-cost predictor, and an algebraic construction, A\* is A-star search, RW is random walk, BF is brute force, dRL is direct reinforcement learning, and forge is the full pipeline.

| Target  | v-rl  | v-girth | v-tabu | v-alg | A*   | RW   | BF   | dRL   | forge |
|---------|-------|---------|--------|-------|------|------|------|-------|-------|
| (5, 3)  | 0.06  | 0.01    | 0.01   | 0.01  | 0.01 | 0.00 |      | 24.64 | 0.04  |
| (6, 3)  | 0.14  | 0.03    | 0.03   | 0.03  | 0.02 | 0.00 |      | 25.07 | 0.16  |
| (7, 3)  | 0.54  | 0.06    | 0.06   | 0.06  | 0.03 | 0.00 |      |       | 0.72  |
| (3, 5)  | 0.02  | 0.16    | 0.19   | 0.01  | 0.03 | 0.01 |      | 23.79 | 0.15  |
| (5, 4)  | 0.06  | 0.02    | 0.02   | 0.02  | 0.03 | 0.03 |      | 34.70 | 0.08  |
| (6, 4)  | 0.10  | 0.04    | 0.04   | 0.04  | 0.09 | 0.10 |      |       | 0.14  |
| (3, 6)  | 0.02  | 0.04    | 0.06   | 0.01  | 0.07 | 0.03 | 0.11 | 56.72 | 0.17  |
| (7, 4)  | 0.16  | 0.07    | 0.07   | 0.07  | 0.18 | 0.24 |      |       | 0.39  |
| (4, 5)  | 0.22  | 2.19    | 2.41   | 0.61  |      |      |      |       | 28.25 |
| (3, 7)  | 0.35  | 37.69   | 46.32  | 5.41  | 7.33 |      |      |       | 39.48 |
| (4, 6)  | 0.06  | 1.38    | 1.61   | 0.91  | 0.94 | 0.53 | 1.45 |       | 14.15 |
| (5, 5)  | 3.86  | 25.32   | 19.82  | 13.62 |      |      |      |       |       |
| (3, 8)  | 0.66  |         |        | 20.47 | 1.16 | 0.78 | 1.69 |       |       |
| (6, 5)  | 20.44 |         |        |       |      |      |      |       |       |
| (5, 6)  | 0.22  | 28.26   | 15.40  | 31.16 | 7.02 | 5.30 |      |       |       |
| (3, 9)  | 0.83  |         |        |       |      |      |      |       |       |
| (7, 5)  |       |         |        |       |      |      |      |       |       |
| (4, 7)  |       |         |        |       |      |      |      |       |       |
| (3, 10) | 1.58  |         |        |       |      |      |      |       |       |
| (6, 6)  | 1.16  |         |        |       |      | 7.96 |      |       |       |
| (4, 8)  | 15.22 |         |        |       |      |      |      |       |       |
| (5, 7)  |       |         |        |       |      |      |      |       |       |

Table 3: Mean time to solve (seconds); blank = unsolved within budget.

| Target  | v-rl    | v-girth | v-tabu | v-alg  | A*     | RW     | BF     | dRL   | forge |
|---------|---------|---------|--------|--------|--------|--------|--------|-------|-------|
| (5, 3)  | 17/42   | 15/37   | 15/37  | 15/37  | 6/15   | 6/15   |        | 8/20  | 8/21  |
| (6, 3)  | 18/53   | 19/57   | 19/57  | 19/57  | 7/21   | 7/21   |        | 9/27  | 9/27  |
| (7, 3)  | 18/64   | 24/82   | 24/82  | 24/82  | 8/28   | 8/28   |        |       | 11/37 |
| (3, 5)  | 20/29   | 15/22   | 15/22  | 15/23  | 10/15  | 10/15  |        | 12/18 | 11/16 |
| (5, 4)  | 17/42   | 15/37   | 15/37  | 15/37  | 10/25  | 10/25  |        | 12/30 | 11/28 |
| (6, 4)  | 20/61   | 19/58   | 19/58  | 19/58  | 12/36  | 12/36  |        |       | 14/44 |
| (3, 6)  | 22/33   | 15/22   | 15/23  | 15/23  | 14/21  | 14/21  | 28/42  | 16/24 | 14/21 |
| (7, 4)  | 24/85   | 23/79   | 23/79  | 23/79  | 14/49  | 14/49  |        |       | 18/63 |
| (4, 5)  | 31/62   | 29/58   | 28/56  | 33/67  |        |        |        |       | 25/50 |
| (3, 7)  | 49/73   | 49/74   | 51/76  | 51/76  | 24/36  |        |        |       | 34/51 |
| (4, 6)  | 42/84   | 30/60   | 27/54  | 27/55  | 26/52  | 26/52  | 52/104 |       | 27/55 |
| (5, 5)  | 48/121  | 54/136  | 46/116 | 63/156 |        |        |        |       |       |
| (3, 8)  | 71/106  |         |        | 56/84  | 30/45  | 30/45  | 60/90  |       |       |
| (6, 5)  | 72/215  |         |        |        |        |        |        |       |       |
| (5, 6)  | 72/180  | 54/136  | 47/119 | 61/153 | 42/105 | 42/105 |        |       |       |
| (3, 9)  | 95/142  |         |        |        |        |        |        |       |       |
| (7, 5)  |         |         |        |        |        |        |        |       |       |
| (4, 7)  |         |         |        |        |        |        |        |       |       |
| (3, 10) | 125/187 |         |        |        |        |        |        |       |       |
| (6, 6)  | 110/331 |         |        |        |        | 62/186 |        |       |       |
| (4, 8)  | 141/283 |         |        |        |        |        |        |       |       |
| (5, 7)  |         |         |        |        |        |        |        |       |       |

Table 4: Mean size of produced graphs,  $|V| / |E|$  (blank = unsolved). Columns as in Table 3.

Voltage-rl has by far the widest reach. It is the only method that solves the high-girth and high-degree targets (3, 9), (3, 10), (4, 8), and (6, 6), and the only one to crack (6, 5), at 57% of seeds. The classical searches, A\* and random walk, are fast and reach cage size on the targets they solve, but only the easy low-girth ones, and they fail entirely past a moderate point. Brute force solves almost nothing. Direct RL solves only trivial targets and is the slowest by far. The voltage variants with GNN guidance, the girth and tabu-cost predictors, collapse earlier than voltage-rl and the algebraic search, so the learned per-node guidance does not extend reach.

On size, the classical methods produce the smallest graphs where they work, the voltage family produces graphs around twice the [Moore bound](#), and the full forge pipeline produces graphs much closer to cage size while solving fewer targets than voltage-rl. No method solves (4, 7), (5, 7), or (7, 5) within budget. For reference, the Moore bounds of (3, 5), (6, 5), (3, 10), and (5, 7) are 10, 37, 62, and 106.

### 9.3 Forge

| Producer          | Success $\uparrow$ | $ V $ / Moore $\downarrow$ |
|-------------------|--------------------|----------------------------|
| voltage-rl        | 0.31               | 1.51                       |
| voltage-girth     | 0.45               | 1.24                       |
| voltage-tabu      | 0.46               | 1.22                       |
| voltage-algebraic | 0.46               | 1.23                       |

Table 5: Forge by voltage producer, full pipeline, over all targets and seeds.

The three tabu-based producers tie at about 46% success and around 1.23 times the Moore bound, all beating the reinforcement-learning producer on both axes. The same voltage-rl policy with the widest standalone reach is the weakest producer inside forge, because the pipeline benefits from a varied stream of near-cage lifts that the more deterministic policy does not supply.

| Configuration | Success $\uparrow$ | $ V $ / Moore $\downarrow$ |
|---------------|--------------------|----------------------------|
| full pipeline | 0.31               | 1.51                       |
| no refinement | 0.62               | 1.58                       |
| no excision   | 0.37               | 2.06                       |

Table 6: Stage ablation on the voltage-rl producer.

The ablations are measured on the voltage-rl producer. Removing excision pushes the size ratio from 1.51 to 2.06, so excision is what shrinks the graph. Removing refinement raises success from 0.31 to 0.62 but enlarges the graphs, so on these targets refinement costs coverage for the rl producer. The per-producer ablations were not run, so this row speaks only for the voltage-rl producer. Forge recovers much of the classical methods’ size optimality, near 1.23 times the Moore bound, while keeping a large part of the voltage family’s reach.



## 10 Future Work

The most promising openings build on the forge cascade of Chapter 8, which the results identify as the most balanced construction method. Its three stages, the producer, the refinement, and the excision, were each tuned in isolation and then chained, so the natural next step is to co-adapt them rather than leave each one optimising a local proxy. The excision stage is the clearest candidate, since it currently sweeps every vertex as a candidate root in a fixed order. A learned policy could instead choose which vertex and to what depth to excise, spending the per-attempt budget where a reduction is most likely. The producer could be trained for diversity instead of single-shot validity, giving excision more varied material to work with. None of this matters unless the cascade can be scaled to the larger targets that no method here reached within budget, which is the clearest measure of whether these refinements pay off.

A second direction is the search for record graphs, which the forge pipeline can already attempt. It has been run as a long, fully parallel search on the PERUN supercomputer against four open targets, the  $(8, 5)$ ,  $(9, 5)$ ,  $(10, 5)$  and  $(11, 6)$   $(k, g)$ -graphs, banking every valid graph it produces by girth so that graphs falling short of the target are still kept as records for the lower girth they reach. To date no record-beating graph has been found. The search remains ongoing.

A third direction concerns how the models are conditioned. Every model in this thesis is asked to handle all degrees and girths at once, but the voltage-lift formulation suggests this asks for more than is needed. A voltage lift is  $k$ -regular by construction, so the degree is never a property a model has to enforce or even observe. What is left to judge is girth, and a fixed  $g$  pins down a rigid local picture: every vertex sits at the centre of a tree of distinct vertices out to radius  $\lfloor (g - 1)/2 \rfloor$  that depends on  $g$  alone. Raising  $k$  widens that tree but leaves its depth unchanged. A model that fixes  $g$  and never sees  $k$  could specialise to exactly the structure that decides girth, and conditioning on the degree at all may prove redundant.

## 11 Conclusion

This thesis asked whether graph neural networks are useful for constructing small  $(k, g)$ -graphs, and if so, where and how. The question was deliberately narrow. A message-passing graph neural network cannot even detect girth in general, and direct reinforcement learning has succeeded on extremal-graph problems mainly where the space of moves already carries structure. The realistic question was therefore not whether a learned model can produce a cage on its own, but whether learning can guide where a search looks while exact algorithms decide which graphs are valid.

The work approached this as a progression of increasing structure. It first probed what graph neural networks can and cannot learn about the two constraints that define a  $(k, g)$ -graph, treating degree as a local counting property and the minimum cycle through a vertex as a global one. It then worked through construction formulations that build in progressively more structure: from unrestricted edge editing under a learned policy, through algebraic voltage lifts in which  $k$ -regularity is structural and only girth must be controlled, to the edge-swap refinement and excision that repair and shrink an already-built graph, and finally to the Forge cascade that composes a voltage producer, refinement, and excision into a single pipeline. At every step the same question was asked of the learned component: whether it actually improves the search over its non-learned counterpart.

On the probing tasks, degree prediction is essentially solved, with a tiny model reaching perfect accuracy, while minimum-cycle prediction resists every architecture tested, with the best model still mislabelling almost half of all vertices. Across the construction methods the dominant pattern is a trade-off between coverage and size. Voltage based searches reach the most targets but returns graphs about twice the Moore bound. The classical searches are size-optimal but narrow, succeeding only on the easier half of the grid. Direct reinforcement learning is the weakest method by a wide margin. Forge is the most balanced, recovering near-cage sizes of roughly 1.27 times the Moore bound on while keeping much of the voltage family’s reach, and meeting the cage exactly on the smallest targets. The largest targets on the grid remained out of reach for every method within the budget tested.

The value of learning here is conditional, and stating that plainly is one of the thesis’s results. The learned voltage producers neither clearly help nor hurt: their success rates and graph sizes track the algebraic search closely, so a learned component that merely matches its non-learned counterpart is reported as exactly that rather than dressed up as a win. The reinforcement-learning voltage policy is the strongest standalone construction method yet the weakest producer inside Forge, because the cascade feeds on a diverse stream of near-cage lifts that a deterministic-leaning policy does not supply. The clearest contributions of

learning are narrower and more specific: the learned repair inside excision, which guides the boundary reconnection while an exact backtracking search guarantees validity, and the role of the reinforcement-learning producer as one useful, though not the best, source of lifts within Forge.

Taken together, these results answer the guiding question. Learning did not replace structured search, and on this problem it rarely beat a well-designed non-learned counterpart outright. Its role was instead to guide and densify the signal inside a search whose validity is still decided by exact algorithms, the same division of labour that runs through every construction method in the thesis, where a learned component ranks or proposes and an exact check accepts. This is the modest, honest position the introduction anticipated: not that a network produces cages by itself, but that learning can sit usefully beside structured search.

Several openings remain, treated at length in Chapter 10 and noted only briefly here. Forge is the clearest candidate for improvement, through co-adapting its stages and learning the excision root and depth rather than sweeping them. The record-graph search already runs the pipeline as a long parallel loop but has not yet beaten a published upper bound on the open cages it targets. And a girth-specialised, degree-agnostic model, one that fixes  $g$  and ties its receptive field to the fixed local radius that the girth singles out, is a hypothesis worth testing against the fully general models used throughout this work.

## Bibliography

- [1] A. Z. Wagner, “Constructions in combinatorics via neural networks.” 2021.
- [2] G. Exoo and R. Jajcay, “Dynamic Cage Survey,” *The Electronic Journal of Combinatorics*, 2013, doi: 10.37236/37.
- [3] G. Exoo, J. Goedgebeur, J. Jookan, L. Stubbe, and T. Van den Eede, “New small regular graphs of given girth: the cage problem and beyond.” 2025.
- [4] M. Grohe, “The Logic of Graph Neural Networks,” in *Proceedings of the 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, 2021, pp. 1–17. doi: 10.1109/LICS52264.2021.9470677.
- [5] V. K. Garg, S. Jegelka, and T. Jaakkola, “Generalization and Representational Limits of Graph Neural Networks,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020, pp. 3419–3430.
- [6] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural Message Passing for Quantum Chemistry,” in *Proceedings of the 34th International Conference on Machine Learning*, 2017, pp. 1263–1272.
- [7] T. N. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” in *International Conference on Learning Representations*, 2017.
- [8] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive Representation Learning on Large Graphs,” in *Advances in Neural Information Processing Systems*, 2017.
- [9] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How Powerful are Graph Neural Networks?,” in *International Conference on Learning Representations*, 2019.
- [10] W. Hu *et al.*, “Strategies for Pre-training Graph Neural Networks,” in *International Conference on Learning Representations*, 2020.
- [11] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, “Recipe for a General, Powerful, Scalable Graph Transformer,” in *Advances in Neural Information Processing Systems*, 2022.
- [12] R. Paolino, S. Maskey, P. Welke, and G. Kutyniok, “Weisfeiler and Leman Go Loopy: A New Hierarchy for Graph Representational Learning,” in *Advances in Neural Information Processing Systems*, 2024.
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms.” 2017.
- [14] A. Lubotzky, R. Phillips, and P. Sarnak, “Ramanujan graphs,” *Combinatorica*, vol. 8, no. 3, pp. 261–277, 1988, doi: 10.1007/BF02126799.

- [15] G. Exoo and R. Jajcay, “On the girth of voltage graph lifts,” *European Journal of Combinatorics*, vol. 32, no. 4, pp. 554–562, 2011, doi: 10.1016/j.ejc.2010.12.003.
- [16] G. Exoo, R. Jajcay, and T. Raiman, “On decreasing the orders of  $(k, g)$ -graphs,” *Journal of Combinatorial Optimization*, vol. 46, p. 26, 2023, doi: 10.1007/s10878-023-01092-9.
- [17] D. Selsam, M. Lamm, B. Bünz, P. Liang, L. de Moura, and D. L. Dill, “Learning a SAT Solver from Single-Bit Supervision,” in *International Conference on Learning Representations*, 2019.
- [18] G. Bouritsas, F. Frasca, S. Zafeiriou, and M. M. Bronstein, “Improving Graph Neural Network Expressivity via Subgraph Isomorphism Counting,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 1, pp. 657–668, 2023, doi: 10.1109/TPAMI.2022.3154319.
- [19] I. Damnjanović, U. Milivojević, I. Đorđević, and D. Stevanović, “RLGT: A reinforcement learning framework for extremal graph theory.” 2026.
- [20] B. C. A. Freire, N. Delfosse, and A. Leverrier, “Optimizing hypergraph product codes with random walks, simulated annealing and reinforcement learning,” in *2025 IEEE International Symposium on Information Theory (ISIT)*, 2025.
- [21] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: theory and application to reward shaping,” in *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, Morgan Kaufmann, 1999, pp. 278–287.
- [22] A. T. Balaban, “Trivalent graphs of girth nine and eleven, and relationships among cages,” *Revue Roumaine de Mathématiques Pures et Appliquées*, vol. 18, pp. 1033–1043, 1973.